
RACE 11 - WEB APP

Bufferdag filter & sortering

Opsamling, afklaring og projektarbejde forud for programmeringen af jeres Web Application-projekt.

13-05-2026

RAMMEN FOR DAGEN

Sådan bruger vi dagen

Vi starter med et kort oplæg om filtrering og sortering. Derefter bruger vi tiden på det, der hjælper jer bedst videre i jeres webapp-projekt.

Kort oplæg

Filtrering og sortering med Supabase, React og JavaScript.

Opsamling

Spørgsmål om React, Supabase, fetch, CRUD eller projektstruktur.

Hvorfor er det relevant?

Når en webapp har dynamisk data, skal brugeren kunne finde det rigtige indhold hurtigt.

Filtrering og sortering er ofte forskellen på "data på en side" og en app, der faktisk kan bruges.

Filtrering kan ske to steder

I en client-server-model kan vi både filtrere og sortere på serveren og på klienten.

Server-side

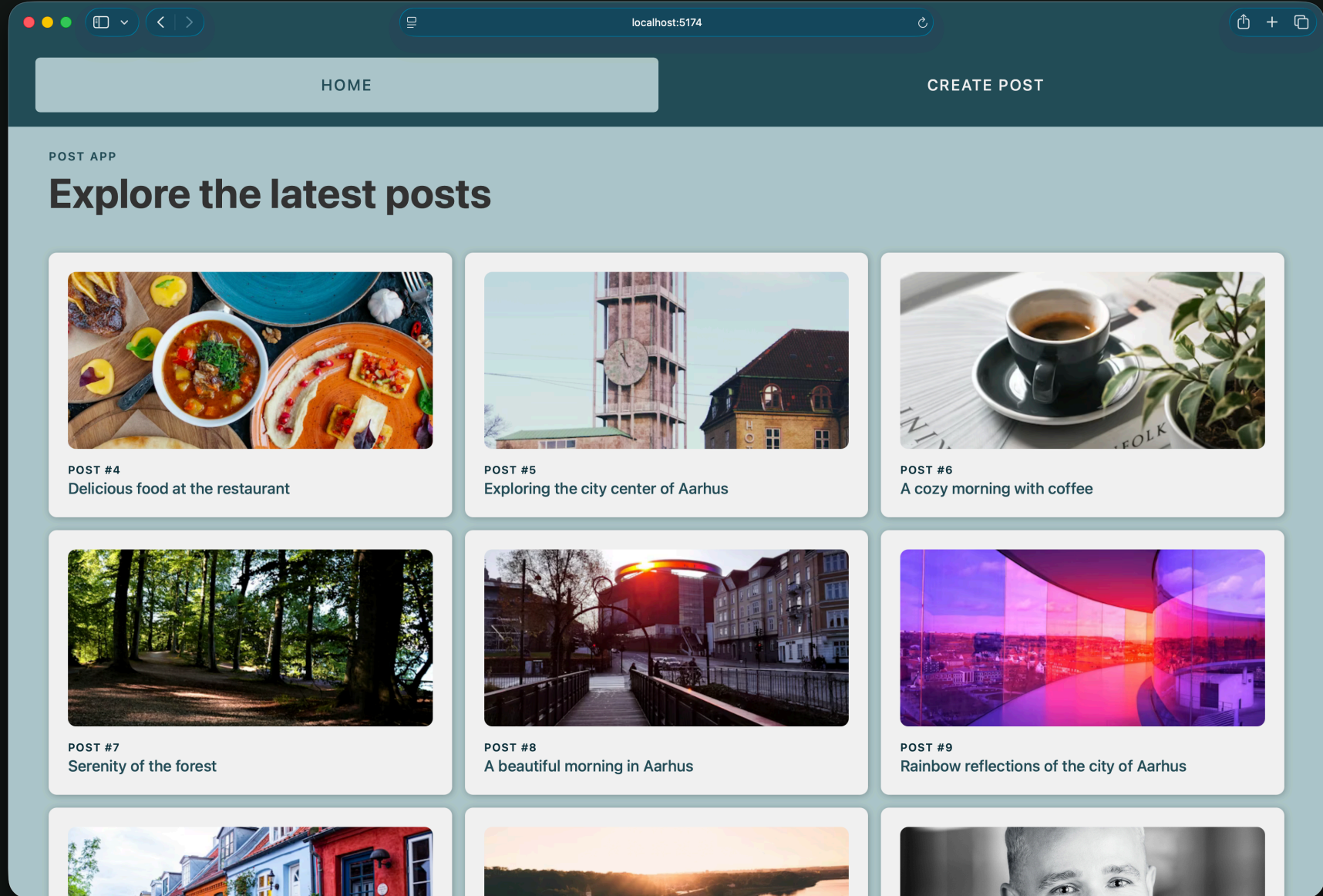
Supabase vælger og sorterer rækkerne, før data sendes til appen.

Client-side

React/JavaScript filtrerer eller sorterer de data, vi allerede har hentet.

KONTEKST

Vi tager udgangspunkt i Post App'en



Det her er data fra Supabase

```
[
  {
    "id": 4,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "Delicious food at the restaurant",
    "image": "https://images.unsplash.com/photo-1548940740-204726a19be3?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 5,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "Exploring the city center of Aarhus",
    "image": "https://images.unsplash.com/photo-1612624629424-ddde915d3dc5?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 6,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "A cozy morning with coffee",
    "image": "https://images.unsplash.com/photo-1545319261-f3760f9dd64d?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 7,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "Serenity of the forest",
    "image": "https://images.unsplash.com/photo-1661505216710-32316e7b5bb3?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 8,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "A beautiful morning in Aarhus",
    "image": "https://images.unsplash.com/photo-1573997953524-efed43db70a0?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 9,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "Rainbow reflections of the city of Aarhus",
    "image": "https://images.unsplash.com/photo-1558443336-dbb3de50b8b2?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 3,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
    "caption": "Exploring the city streets of Aarhus",
    "image": "https://images.unsplash.com/photo-1559070169-a3077159ee16?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1pYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=500&q=80"
  },
  {
    "id": 2,
    "created_at": "2026-04-23T06:19:39.293582+00:00",
  }
```

POST APP

Det data vi tager udgangspunkt i

I Supabase arbejder vi med tabellen `posts`.

`id`

Unikt id. Bruges til detail view, update og delete.

`created_at`

Tidspunktet posten blev oprettet. God til nyeste/først eller ældste/først.

SUPABASE REST

Filter sker med query parameters

Supabase bruger query parameters til at filtrere direkte i databasen.



REST

```
GET /rest/v1/posts?id=eq.1
```

```
GET /rest/v1/posts?caption=ilike.*aarhus*
```

```
GET /rest/v1/posts?id=gt.5
```

Pointen: Vi henter kun de data, vi faktisk skal bruge.

FILTER-EKSEMPLER

Hent ét bestemt post

Det her mønster bruger I allerede på detail-siden.

```
GET /rest/v1/posts?id=eq.4
```

```
GET /rest/v1/posts?id=eq.16
```

REST

eq betyder equals: kolonnen skal være lig med værdien.

FILTER-EKSEMPLER

Søg i captions

ilike betyder case-insensitive **like**: vi filtrerer en tekstkolonne ved at matche et mønster uden forskel på store og små bogstaver.

```
GET /rest/v1/posts?caption=ilike.*aarhus*
```

```
GET /rest/v1/posts?caption=ilike.*coffee*
```

```
GET /rest/v1/posts?caption=ilike.*city*
```

I praksis bruger vi ofte ***** på begge sider, når brugeren bare skal kunne søge efter tekst, der indgår et sted i caption.

Sådan læses **ilike**

Selve filteret er **ilike**. Stjernerne bestemmer, hvor i teksten matchet må ligge.

ilike.aarhus

Caption skal være præcis "aarhus", men store og små bogstaver er ligegyldige.

ilike.aarhus*

Caption skal starte med "aarhus".

ilike.*aarhus

Caption skal slutte med "aarhus".

FILTER-EKSEMPLER

Filtrer på id

Id kan bruges til at hente nyere/oprettede rækker i en simpel demo.

```
GET /rest/v1/posts?id=gt.5
```

```
GET /rest/v1/posts?id=lt.8
```

```
GET /rest/v1/posts?id=gte.15
```

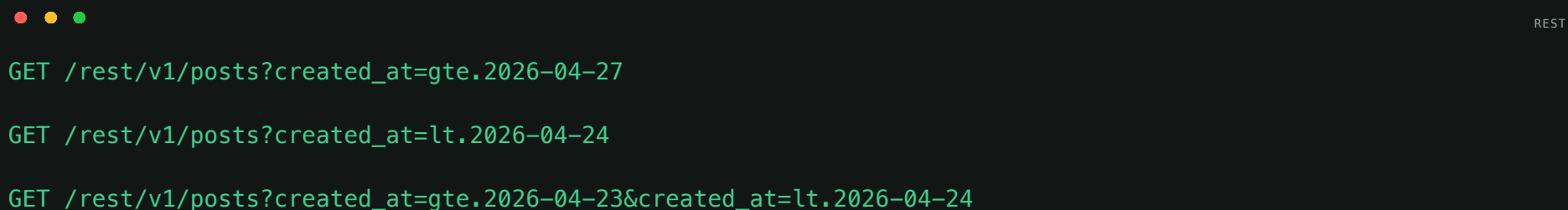
REST

gt, **lt** og **gte** betyder greater than, less than og greater than or equal.

FILTER-EKSEMPLER

Filtrer på dato

Dato-filter er nyttigt, når I vil vise nye posts eller indhold fra en bestemt periode.



```
GET /rest/v1/posts?created_at=gte.2026-04-27  
GET /rest/v1/posts?created_at=lt.2026-04-24  
GET /rest/v1/posts?created_at=gte.2026-04-23&created_at=lt.2026-04-24
```

Datoer kan filtreres med de samme operatorer som tal.

FILTER-EKSEMPLER

Kombiner flere filtre

Flere query parameters sættes sammen med **&**.

```
GET /rest/v1/posts?caption=ilike.*aarhus*&id=gt.4
```

```
GET /rest/v1/posts?caption=ilike.*city*&created_at=gte.2026-04-23
```

```
GET /rest/v1/posts?caption=ilike.*aarhus*&order=created_at.desc&limit=3
```

Når query'en bliver lang, er **URLSearchParams** en god ven i React.

OPERATORER

De mest brugbare i Post App

eq

Lig med. Bruges allerede i detail view: `?id=eq.1`.

ilike

Case-insensitive tekstfilter. Sammen med `*` kan det matche præcis tekst, start, slut eller indhold.

SORTERING

Sortering sker med **order**

order fortæller først hvilken kolonne vi sorterer på, og derefter retningen: **asc** eller **desc**.

```
GET /rest/v1/posts?order=created_at.desc
```

```
GET /rest/v1/posts?order=created_at.asc
```

```
GET /rest/v1/posts?order=caption.asc
```

```
GET /rest/v1/posts?order=caption.desc
```

created_at.desc betyder fx: sorter efter **created_at** i faldende rækkefølge.

Sådan læses **asc** og **desc**

asc

Ascending. Laveste til højeste værdi, A til Z eller ældste til nyeste.

desc

Descending. Højeste til laveste værdi, Z til A eller nyeste til ældste.

created_at.desc

Nyeste posts først, fordi de seneste datoer ligger øverst.

caption.asc

Caption sorteres alfabetisk fra A til Z.

caption.desc

Caption sorteres alfabetisk fra Z til A.

Mønsteret er: **order=kolonne.retning.**

KOMBINATION

Filter, sortering og limit kan kombineres



```
GET /rest/v1/posts?caption=ilike.*aarhus*&order=created_at.desc&limit=5
```

REST

Læs URL'en som en sætning: posts hvor caption indeholder "aarhus", nyeste først, maks 5 resultater.

JAVASCRIPT

Det er stadig bare et **fetch**

```
1  const response = await fetch(  
2    "https://xyz.supabase.co/rest/v1/posts?caption=ilike.*aarhus*&order=created_at.desc&limit=5",  
3    {  
4      headers: {  
5        apikey: "din_sb_publishable_xyz"  
6      }  
7    }  
8  );  
9  
10 const data = await response.json();
```

JAVASCRIPT

REACT

UI styrer query'en

I React kommer filter og sortering typisk fra state: inputfelter, dropdowns, knapper eller tabs.

```
1  const searchText = "aarhus";
2  const sort = "created_at.desc";
3
4  const query =
5    `?caption=ilike.*${searchText}*&order=${sort}`;
6
7  const url = `${URL}${query}`;
```

REACT

HOMEPAGE

Fra alle posts til filtrerede posts

```
1  useEffect(() => {  
2    async function getPosts() {  
3      const response = await fetch(URL, { headers });  
4      const data = await response.json();  
5      setPosts(data);  
6    }  
7  
8    getPosts();  
9  }, []);
```

HOMEPAGE.JSX

Det her er jeres nuværende udgangspunkt: hent alle posts.

HOMEPAGE

Samme fetch, men med query

```
1  const query =  
2    `?caption=ilike.*${searchText}*&order=${sort}`;  
3  
4  const response = await fetch(`${URL}${query}`, { headers });  
5  const data = await response.json();  
6  setPosts(data);
```

HOMEPAGE.JSX

Det nye er ikke `fetch`. Det nye er URL'en.

GOD VANE

Byg query'en mere sikkert

```
1  const params = new URLSearchParams();
2
3  if (searchText.trim()) {
4    params.set("caption", `ilike.*${searchText.trim()}*`);
5  }
6
7  params.set("order", sort);
8
9  const response = await fetch(`${URL}?${params}`, { headers });
```

HOME PAGE.JSX

Det gør det lettere at tilføje flere filtre senere.

Sådan hænger det sammen i React

1. Input

Brugeren skriver "aarhus" eller vælger en sortering.

2. State

`searchText` eller `sort` ændrer sig.

3. Fetch

`useEffect` kører igen, fordi dependency array lytter på ændringer i `searchText` og `sort`.

REACT-STATE

State holder styr på filter og sortering

```
1 export default function HomePage() {  
2   const [posts, setPosts] = useState([]);  
3   const [searchText, setSearchText] = useState("");  
4   const [sort, setSort] = useState("created_at.desc");  
5  
6   // ...  
7 }
```

HOME PAGE.JSX

Når `searchText` eller `sort` ændrer sig, kan vi hente posts på en ny måde.

CONTROLLED INPUT

Brugeren ændrer state

```
1 <input type="search" placeholder="Search captions..." value="{searchText}" onChange="{(event)" ==""> setSear
2 />
3
4 <select value="{sort}" onChange="{(event)" ==""> setSort(event.target.value)}
5 >
6   <option value="created_at.desc">Newest first</option>
7   <option value="created_at.asc">Oldest first</option>
8   <option value="caption.asc">Caption A-Z</option>
9   <option value="caption.desc">Caption Z-A</option>
10 </select>
```

UI'et viser altid state, og brugerens handlinger opdaterer state.

USEEFFECT

State ændrer query'en

```
1  useEffect(() => {
2    async function getPosts() {
3      const params = new URLSearchParams();
4
5      if (searchText.trim()) params.set("caption", `ilike.*${searchText.trim()}*`);
6      params.set("order", sort);
7
8      const response = await fetch(`${URL}?${params}`, { headers });
9      setPosts(await response.json());
10   }
11
12   getPosts();
13 }, [searchText, sort]);
```

`useEffect` er nyttig her, fordi dependency array får den til at køre igen, når `searchText` eller `sort` ændrer sig.

RESET

Nulstil filteret

```
1 function resetFilters() {  
2   setSearchText("");  
3   setSort("created_at.desc");  
4 }  
5  
6 <button onclick="{resetFilters}">  
7   Reset filters  
8 </button>
```

JSX

Når state nulstilles, kører `useEffect` igen og henter den oprindelige liste.

Filtrering kan stadig ske i JavaScript

```
1  const filteredPosts = posts.filter((post) =>
2    post.caption.toLowerCase().includes(searchText.trim().toLowerCase())
3  );
4
5  // ...
6  {filteredPosts.map((post) => (
7    <postcard key="{post.id}" post="{post}">
8  ))}
9    </postcard>
```

HOMEPAGE.JSX

I client-side versionen henter `useEffect` alle posts én gang, og derefter filtrerer vi dem lokalt i React.

Sortering kan også ske lokalt

```
1 filteredPosts.sort((postA, postB) => {
2   if (sort === "created_at.desc") {
3     return new Date(postB.created_at) - new Date(postA.created_at);
4   }
5
6   if (sort === "created_at.asc") {
7     return new Date(postA.created_at) - new Date(postB.created_at);
8   }
9
10  if (sort === "caption.desc") {
11    return postB.caption.localeCompare(postA.caption);
12  }
13
14  return postA.caption.localeCompare(postB.caption);
15 });
```

HOMEPAGE.JSX

Her sorterer vi direkte på `filteredPosts`, fordi `filter()` allerede har lavet et nyt array.

BEGGE DELE VIRKER

Lokal logik vs. filter i Supabase

Lokalt i React

Godt når data allerede er hentet, og listen er lille eller mellemstor.

I Supabase query

Godt når databasen selv skal finde de rigtige rækker, før de sendes til appen.

Fordel lokalt

MINI-DEMO

Det vi kan teste sammen

- En version hvor `useEffect` henter alle posts én gang.
- Et tekstfelt der filtrerer lokalt på `caption`.
- En select der sorterer efter `created_at` eller `caption` i begge retninger.
- En knap der nulstiller filteret.
- At filtrering og sortering bagefter sker lokalt i JavaScript.

VALG EFTER OPLÆGGET

Hvad gør vi nu?

Fælles opsamling

Vi tager spørgsmål i plenum og samler de vigtigste pointer.

Git/GitHub/deploy

Vi arbejder med repo, branches, PRs, GitHub Pages og environment variables.

Gruppevejledning

Jeg går rundt og hjælper grupperne videre der, hvor I står.

PROJEKTBLIK

Inden I programmerer for meget

- Hvilke centrale user flows vil I programmere?
- Hvilke data skal gemmes i Supabase?
- Hvilke felter skal brugeren kunne søge, filtrere eller sortere på?
- Hvilke komponenter går igen flere steder?
- Hvordan kan alle i gruppen bidrage tydeligt med commits?
- Hvad skal være klar til deployment?

MATERIALER

Links til i dag

- Post App med Supabase
- Branch: filter-server-side (Supabase/server-side)
- Branch: filter-client-side (React/client-side)
- React Router SPA Template
- Fra Template til GitHub Pages
- Samarbejdsguide
- Deploy your React App to GitHub Pages

ARBEJD VIDERE

Brug dagen på det, der flytter jeres projekt

Spørg, test, afklar, commit, prototype, programmer - og få
hjælp, når I sidder fast.

Hvad er jeres næste bedste skridt?